

PROJECT AND TEAM INFORMATION

Project Title

Smart Public Complaint Analyzer

Student / Team Information

Team Name: <i>DSCPP-III-2026-T031</i>	<i>CivicSync</i>
Team member 1 (Team Lead)	<i>Rawat, Devraj –</i> <i>2510370241</i> <i>2510370241@geu.ac.in</i> 
Team member 2	<i>Mahajan, Stuti –</i> <i>2510010881</i> <i>2510010881@geu.ac.in</i> 
Team member 3	<i>Panwar, Archi –</i> <i>2510012717</i> <i>2510012717@geu.ac.in</i> 
Team member 4	<i>Rajat – 2510370572</i> <i>2510370572@geu.ac.in</i> 

PROPOSAL DESCRIPTION

Motivation

Ever felt that the complaints about a local civil issue such as the broken street lamp, large hole in the road, or trash not being removed are not handled properly? When offices get hundreds of reports all at once, workers struggle to find what work he needs to do first, to come up with this we created the Smart Public Complaint Analyzer. When someone reports a local issue like a broken street light or a pothole the system saves all the details and send it to the admin, based on how important it is. The system allows the administrator to search and prioritize complaints as required and routes the complaint to the department with logs and updates on the current status. Essentially, not all public issues are treated the same, and there are some that need to be addressed with higher priority than others. Some complaints do not require a high-priority response, and some are even duplicates, thus wasting time and effort. A well managed digital database makes it easy to prioritize and route complaints to the appropriate department, ensuring that no time and effort are wasted on unnecessary tasks. From an engineering perspective, the Smart Public Complaint Analyzer requires the integration of low-level data structures and advanced application development. For the former, the C programming language(engine for the program) implements such as arrays, linked lists, queues, priority queues, hash tables, trees, graphs to manage the storage, retrieval, and update of complaint data. For the latter, C++ (for user interface) features such as classes, inheritance, polymorphism, abstraction, exception handling, the STL, and file handling are used to develop the application, configure the objects and classes of users and administrators, and provide an interface for them to interact with the system.

State of the Art / Current solution

Currently, public complaints are being managed using manual process Citizens report complaints like damaged roads, garbage collection, street-light failure, or water-logging through various avenues. And as the number of complaints increases, it becomes difficult to manage them all manually. These complaints often require manual recording, categorizing, searching, prioritizing, and forwarding to the particular department, which can lead to duplicates, incorrect routing, delays in urgent cases, and difficulty in finding the status of a complaint. Admins and departments may also have to keep separate records and track the complaints progress independently. So searching for a complaint, or reviewing the history of complaints, can become time-consuming when having too many records. So the current approach is not efficient enough in terms of having a unified mechanism to store, organize, prioritize, search, and route the complaints. therefore, Proposed Smart Public Complaint Analyzer which addresses all the shortcomings by using data structure like queues, priority queues, hashing, trees, and graphs for complaint processing and routing, while C++ OOP concepts are used to manage users, administrators, departments, and employees.

Project Goals and Milestones

The main goal of the Smart Public Complaint Analyzer is to develop a structured system that efficiently manages public complaints from registration to resolution. The project aims to demonstrate how C data structures and C++ object-oriented programming can work together to solve a real-world problem.

The general goals of the project are:

To provide a systematic method for registering and storing citizen complaints. To organize complaints according to their category, severity, priority, and status. To efficiently search and sort complaints for faster processing. To prioritize urgent complaints so that high-severity issues can be handled first. To automatically route complaints to the appropriate department using suitable data structures. To maintain complaint history and track the resolution status. To provide an administrator dashboard for viewing, assigning, and monitoring complaints. To demonstrate the practical application of arrays, linked lists, queues, priority queues, hashing, BSTs, graphs, and sorting/searching algorithms. To apply C++ concepts such as classes, inheritance, polymorphism, abstraction, exception handling, STL, and file handling.

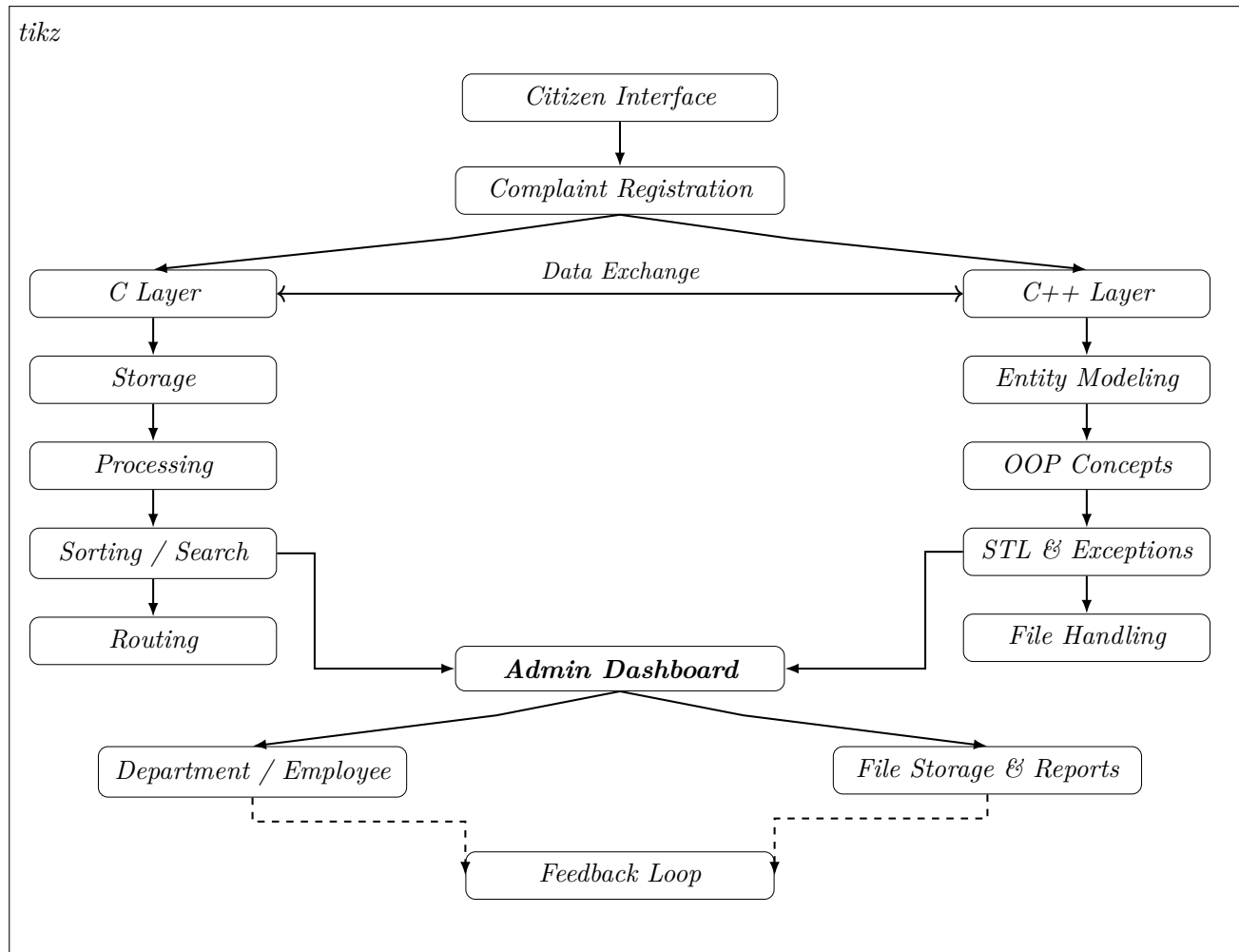
Initial Milestones Finalize the system requirements and overall workflow. Design the complaint structure and basic registration module. Implement complaint storage using arrays and linked lists. Develop queue and priority-queue based complaint processing. Implement sorting, searching, hashing, BST, and graph-based routing.

Further Milestones Develop C++ classes for users, administrators, departments, employees, and complaints. Integrate file handling, STL, exception handling, and reporting. Develop the integrated administrator dashboard. Combine the C data-structure engine with the C++ OOP layer. Test the complete system, generate reports, and prepare the final demonstration.

Project Approach

The proposed solution will be designed as a modular system in which C acts as the internal data-structure engine and C++ acts as the object-oriented application layer(UI). This approach will allow the project to demonstrate both data structures and OOP concepts while solving the complaint-management problem. First, a Complaint structure will be designed to store important information such as complaint ID, citizen details, description, category, location,timestamp and if a new user enter he needs to sign up first to file a complaint . C will then be used to implement the core operations for storing, processing, searching, sorting, prioritizing, and routing complaints.We will be using different data structures according to their purpose. Arrays and linked lists willmanage complaint records, queues and priority queues will handle complaint processing and urgency,while hashing will assist in department mapping. BSTs will support complaint indexing and searching,and graphs with BFS/DFS will model department escalation and routing. The C++ layer will model real-world entities such as User, Admin, Department, Employee, andComplaint using classes. Concepts including encapsulation, inheritance, polymorphism, abstraction, operator/function overloading, exception handling, templates, and STL will be incorporated. Filehandling will be used to save and load complaint records, while an administrator dashboard will integrate the major components and generate reports. Platforms and Technologies Programming Languages: C and C++ Development Platform: Desk-top/PC environment Libraries: C standard libraries and C++ STL Data Storage: File handling using C++ fstream Interface: Menu-driven administrator dashboard Core Technologies: Data Struc-tures, Algorithms, OOP, File I/O, STL, and Exception Handling

System Architecture (High Level Diagram)



Project Outcome / Deliverables

The primary outcome of the project will be a functional Smart Public Complaint Analyzer that provides a structured mechanism for handling public complaints from registration to resolution. The system will demonstrate the practical application of C data structures and C++ OOP concepts in a real-world problem.

The major deliverables of the project will include:

A Complaint Registration Module for recording complaint details such as ID, description, category, location and timestamp. A C-based Data Structure Engine implementing storage, processing, sorting, searching, prioritization, and routing of complaints. A C++ OOP Application Layer containing classes for users, administrators, departments, employees, and complaints. An Admin Dashboard for viewing, assigning, tracking, and managing complaints. File handling functionality for saving, loading, and maintaining complaint history. Department/Employee Module for receiving assigned complaints and updating their resolution status. Reports and Analytics for pending, resolved, categorized, and department-wise complaints. A feedback/status update mechanism to maintain the complaint lifecycle.

Assumptions

The project assumes that citizens provide accurate and sufficient information while submitting complaints. Each complaint is assigned a unique ID and contains details such as description, category, priority, and timestamp. It is assumed that complaints can be categorized into predefined departments and that departments/employees are available to handle assigned complaints. The system is designed for a controlled set of complaint records and uses file-based storage rather than a live database. It is also assumed that administrators are responsible for assigning, monitoring, and updating complaint status

References

- 1.cppreference – C Language Reference — Used for C language syntax, arrays, structures, functions, pointers, memory management, and standard library concepts.*
- 2.cppreference – C++ Language Reference — Used for classes, inheritance, polymorphism, exception handling, templates, and other C++ concepts.*
- 3.cppreference – C++ Standard Library — Used for STL containers, algorithms, iterators, and file/input-output facilities.*
- 4.cppreference – C++ Book — Used as a supplementary reference for classes, encapsulation, inheritance, polymorphism, containers, algorithms, and exceptions.*
- 5. Rana et al. — Adoption of Online Public Grievance Redressal System in India — Study of online grievance systems and citizen adoption. Sabari et al. — Smart Civic Complaint Management System — Focuses on complaint registration, department assignment, and tracking. Parab et al. — Citizens Corner: Grievance Registration and Tracking System — Covers complaint registration, tracking, and grievance resolution.*